

Architectural Analysis of Rust-Based Markdown Parsing Engines for Sandboxed WYSIWYG Environments

The evolution of modern WYSIWYG Markdown editors demands a fundamental shift in underlying parsing architectures. Transitioning from traditional single-pass batch renderers to real-time, interactive editing surfaces necessitates engines capable of maintaining stateful, source-authoritative representations of text. For the MarkText editor—an application operating across an Electron and Vue frontend—the objective is to replace a custom TypeScript engine, designated as "document-core," with a highly performant Rust-based alternative. This transition promises significant advantages in memory safety and computational speed, specifically by leveraging WebAssembly (WASM). However, it also imposes an exceptionally stringent set of architectural constraints.

This comprehensive analysis evaluates open-source Rust parsing engines against these immutable requirements: source-authoritative lossless mapping (Concrete Syntax Tree architectures), sub-millisecond incremental reparsing, intrinsic CriticMarkup support, exhaustive CommonMark and GitHub Flavored Markdown (GFM) coverage, mid-edit error tolerance, and WASM/Electron embeddability under MIT-compatible licensing.

Architectural Imperatives and Parsing Theory

To rigorously evaluate the candidate Rust engines, it is necessary to first deconstruct the underlying computer science constraints imposed by a modern, source-authoritative WYSIWYG editor. Standard Markdown parsers are engineered as compilers; their primary function is to translate source text into a final output format, typically HTML. WYSIWYG editors, conversely, operate more akin to language servers. They must maintain a persistent, bidirectional, and perfectly lossless representation of the document state that maps synchronously to a visual Document Object Model (DOM).

Source-Authoritative and Lossless Parsing: The AST vs. CST Dichotomy

Traditional parsing architectures generate an Abstract Syntax Tree (AST), which intrinsically strips away non-semantic trivia such as whitespace, formatting characters, and syntactic markers to optimize for semantic traversal. For instance, parsing a bold string like **bold** yields a Strong node containing a Text("bold") child, entirely discarding the literal asterisk characters in the memory representation. In a source-authoritative WYSIWYG editor, the original byte-span is the single source of truth. If the editor alters the visual representation of the bold text, the underlying engine must know exactly where the asterisks reside to mutate the source text seamlessly.

This requirement mandates a Concrete Syntax Tree (CST) or a lossless Red-Green tree architecture, wherein every byte—including soft breaks, trailing spaces, and delimiter runs—is accounted for within the tree's memory model. Attempting to reconstruct an AST back into

Markdown invariably leads to normalization (e.g., standardizing all unordered list markers to -), which violates the principle of lossless round-tripping. A true replacement for "document-core" must never guess the user's formatting intent; it must map it byte-for-byte.

Incremental Reparsing and the WASM Boundary

In a keystroke-driven Electron environment, parsing the entire document from scratch on every mutation (batch parsing) yields catastrophic performance degradation as the document size scales. Parsing must be incremental. The engine must possess the capability to isolate the modified text fragment, repair the localized syntax tree, and preserve the unmodified branches in memory. This fragment-reuse methodology, popularized by parsers like Tree-sitter and Lezer, ensures that the time required to parse a keystroke remains constant ($O(1)$ or $O(\log N)$) rather than scaling linearly with the size of the document.

Furthermore, the boundary between the V8 JavaScript engine (executing Vue) and the WASM runtime imposes a severe serialization penalty. Crossing this boundary requires copying memory. If an engine forces the serialization of the entire syntax tree into JSON upon every keystroke, the latency will easily exceed the strict 16-millisecond budget required for 60fps typing, entirely neutralizing the raw speed advantages of Rust. The optimal engine must retain the syntax tree within the WASM linear memory and provide a bridging API for the JavaScript thread to query tree diffs or localized nodes using pointer references.

The CriticMarkup Grammatical Concurrency Problem

CriticMarkup introduces powerful editorial annotations: `{++addition++}`, `{--deletion--}`, `{~old~>new~}`, `{==highlight==}`, and `{>>comment<<}`. Implementing these natively inside a Markdown parser is exceptionally complex because Markdown is context-sensitive and historically parsed in two distinct, sequential phases: a block-level structural analysis (identifying paragraphs, blockquotes, and lists) followed by an inline-level lexical analysis (resolving emphasis, links, and text formatting).

CriticMarkup inherently violates this strict boundary. A deletion marker `{--` may begin in the middle of one paragraph, traverse an intervening blank line, encompass an entire blockquote, and terminate `--}` in a subsequent paragraph. If a parser strictly evaluates block constraints first, it will fail to recognize the multi-block inline span of the CriticMarkup syntax. Because a second-pass sidecar scanner is explicitly forbidden by the MarkText architectural requirements, the candidate engine must possess an extensible grammar that treats CriticMarkup as a first-class, intrinsic syntactic construct capable of interrupting or wrapping standard structural blocks natively.

Mid-Edit Error Tolerance

Markdown, by its specification, has no formal concept of "syntax errors" due to its fallback to plain text. However, extended syntaxes such as LaTeX math blocks, GFM tables, and CriticMarkup introduce strict delimiter pairing requirements. When a user types the opening braces of a CriticMarkup block `{++`, the document enters a technically invalid state until the closing `++}` is typed. An editor-grade parser must generate a usable, error-tolerant tree—tagging the broken node as an ERROR or incomplete state—rather than halting execution, entering an infinite loop, or dropping the node entirely. This resilience ensures the surrounding document remains visually stable during mid-edit typing.

Architectural Paradigm	Memory Representation	Modification Strategy	Error Handling Profile	Suitability for WYSIWYG
Event Stream (Pull)	Transient token yields	Full document re-traversal	Native text fallback	Disqualified
Abstract Syntax Tree	Semantic node hierarchy	Full document batch re-parse	Node dropping	Disqualified
Concrete Syntax Tree (GLR)	Exhaustive byte-mapped nodes	Graph-based incremental re-parse	First-class ERROR nodes	Highly Viable
Red-Green Trees	Immutable deduplicated nodes	Pointer swapping / fragment reuse	Error-resilient node wrapping	Highly Viable

Prior Art: Rust Implementations and CriticMarkup

Before evaluating specific parsing engines, it is highly instructive to analyze existing Rust applications that have integrated CriticMarkup, as they demonstrate the viability of handling the syntax within the Rust ecosystem.

The Quoin editor is a native macOS WYSIWYG Markdown application written primarily in Rust and Swift. It utilizes an in-file review loop where suggestions, comments, and tracked changes are stored as plain CriticMarkup bytes. Quoin guarantees a byte-lossless, local-only environment without relying on JavaScript, proving that a Rust core can successfully maintain CriticMarkup state authority.

Similarly, the Marko Editor is a WYSIWYG editor for note-taking written in Rust and GTK 4. It utilizes Markdown combined directly with CriticMarkup as its diffable storage format, rendering colors for special highlights. However, Marko Editor explicitly notes in its development status that its undo/redo stack lacks formatting awareness and that formatting operates optimally on existing text rather than dynamically while typing. This limitation highlights the exact architectural hurdle MarkText is attempting to solve: batch-processing CriticMarkup is achievable, but incremental, real-time keystroke processing requires a much more sophisticated internal tree structure.

Other tools, such as casile, leverage Rust to transform simple inputs into finished artifacts, utilizing CriticMarkup to syntax-highlight differences (additions and removals) between git branches down to the character level during PDF output generation. While these implementations validate Rust's capability to process CriticMarkup lexically, they are predominantly batch-processors or native GUI applications, differing significantly from the sandboxed, incremental WASM environment required by MarkText.

Application / Crate	Primary Focus	CriticMarkup Usage	Relevance to MarkText
Quoin	macOS Native WYSIWYG	Plain byte-lossless tracking	Proves Rust can handle lossless CriticMarkup
Marko Editor	GTK4 WYSIWYG Notes	Diffable storage format	Highlights keystroke editing limitations
Casile	Artifact Generation CLI	Diff highlighting for PDFs	Batch processing only; no incremental engine
obsidian-track-changes	Obsidian Plugin	Sidebar review mechanism	Operates in TS/JS, utilizing Rust for

Application / Crate	Primary Focus	CriticMarkup Usage	Relevance to MarkText
			backend logic

Evaluation of Pull and Push Event Stream Parsers

Event stream architectures iterate over text linearly, yielding tokens as they are discovered. While highly memory-efficient for compiler pipelines, they fundamentally clash with the requirements of a spatial WYSIWYG editor.

pulldown-cmark

pulldown-cmark is the most widely utilized Markdown parser in the Rust ecosystem. It is designed as a pull parser that yields a stream of discrete events, optimized for zero-allocation performance and strict adherence to the CommonMark specification. It supports generating source-map information by mapping parsed blocks to offsets within the source text via an `into_offset_iter()` function.

Despite its speed and widespread adoption in tools like mdBook and mdcat, pulldown-cmark fundamentally fails the architectural requirements. It is not a lossless engine; it emits a stream of semantic events, stripping delimiters and trivia. Reconstructing the source requires third-party utilities like pulldown-cmark-to-cmark, which normalize the output and break lossless round-tripping. Furthermore, it is a forward-only iterator, meaning it cannot incrementally repars a localized fragment without traversing the entire string from the beginning. Extending the grammar to natively support CriticMarkup is highly problematic because the core state machine must be deeply modified to intercept and yield new event types. While it boasts excellent GFM support (tables, task lists, strikethrough, footnotes) and an MIT license, its lack of a persistent tree structure and absence of incremental parsing immediately disqualify it as a wholesale replacement for the "document-core" engine.

jotdown

jotdown operates on a similar pull-parser architecture but specifically targets the Djot markup language rather than CommonMark. Djot was designed to eliminate many of Markdown's complex heuristics—such as backtracking for emphasis disambiguation—in favor of strict, linear parsing. While jotdown is an exceptionally fast event stream, it parses a fundamentally different specification. It is instantly disqualified as it fails the non-negotiable CommonMark and GFM compliance requirement.

Evaluation of AST-Based and Batch Parsing Engines

AST-based engines parse the entirety of the text into a hierarchical, in-memory tree of nodes. While closer to the structural requirements of an editor than event streams, they traditionally sacrifice trivia and structural fidelity in favor of semantic clarity.

comrak

comrak is a highly accurate, direct Rust port of GitHub's cmark-gfm, offering 1:1 parity with GitHub Flavored Markdown. It constructs an arena-allocated Abstract Syntax Tree and boasts extensive configuration options. The ecosystem includes comrak-wasm, a wrapper providing a

mature, high-level TypeScript API for WebAssembly embedding.

However, `comrak` produces an AST that abstracts away literal byte representations, prioritizing semantic correctness over syntactic fidelity. It is fundamentally disqualified because its arena-allocated AST is generated in a single batch pass; modifying the tree requires a complete re-parse of the source document, violating the incremental reparse requirement. Furthermore, adding intrinsic CriticMarkup would require delving into its heavily C-inspired Rust port architecture to add new AST node types and modify the core block/inline parsing loops, which are not designed for extensibility. It operates under a BSD-2-Clause license, which is MIT-compatible, but its batch architecture is fatal for keystroke-driven latency.

markdown-it.rs

A Rust port of the highly popular JavaScript `markdown-it` library, `markdown-it.rs` attempts to bring the extensible plugin architecture of its predecessor to the Rust ecosystem. It supports source mapping via byte offsets, and its architecture is explicitly designed for arbitrary syntactic expansion.

While a custom CriticMarkup plugin could theoretically be authored for `markdown-it.rs`, the parser remains fundamentally an AST, not a CST. Trivia and whitespace variations are not structurally retained. More critically, it processes the document via sequential core, block, and inline rule chains, requiring full batch execution on every edit. It completely lacks incremental reparsing. Furthermore, the project is explicitly classified as an alpha/beta tech preview, indicating low maintenance health for a production-grade editor core.

markdown-rs (woorm) and mdxjs-rs

Authored by the creator of the `unifiedjs` ecosystem, `markdown-rs` is a meticulously engineered, 100% CommonMark compliant state machine. Unlike standard AST parsers, it accounts for every single byte, outputting concrete tokens mapped with positional data before assembling its final AST (`mdast`). It effectively acts as a CST tokenizer. It boasts exceptional support for GFM, YAML frontmatter, math, and MDX, with `mdxjs-rs` serving as a mature wrapper for MDX compilation.

Despite its theoretical brilliance at tracking byte provenance, `markdown-rs` is disqualified due to severe performance characteristics under specific conditions and its rigid extensibility model. The parser relies on a complex `edit_map` system that has demonstrated asymptotic complexity failures, triggering quadratic $O(N^2)$ time consumption on specific long-document edge cases, resulting in massive parse times. It does not natively support true tree-fragment reuse for sub-millisecond incremental updates. Furthermore, it operates as a closed, monolithic crate without a public plugin or extension API. Supporting CriticMarkup natively would require maintaining a hard fork and undertaking a profound modification of the internal `state.rs` machine, a prohibitive engineering burden.

Engine	Parse Model	Source Fidelity	Incrementality	Error Tolerance
pulldown-cmark	Event Stream	Destructive (No CST)	None (Batch iterator)	Low (Fallback to text)
comrak	AST	Semantic (No CST)	None (Batch arena)	Low
markdown-it.rs	AST	Byte-mapped	None (Batch rules)	Medium
markdown-rs	State Machine /	Token-level exact	Flawed (Quadratic	High

Engine	Parse Model	Source Fidelity	Incrementality	Error Tolerance
	AST		risks)	

Evaluation of Incremental CST Architectures

To satisfy the dual requirements of sub-millisecond keystroke latency and exact byte-span lossless mapping, the architecture must utilize a generalized incremental parsing framework. There are two primary paradigms in the Rust ecosystem capable of this: Generalized Left-to-Right Rightmost derivation (GLR) via tree-sitter, and Red-Green immutable syntax trees via rowan or Typst's incremental parser architectures.

tree-sitter-markdown and tree-sitter-quarto

tree-sitter is a parser generator tool and incremental parsing library designed explicitly for modern text editors. Implementing Markdown in Tree-sitter has historically been fraught due to block/inline context conflicts. However, implementations like tree-sitter-markdown and tree-sitter-quarto bypass this by utilizing a dual-grammar architecture: one grammar evaluates the block structure, and a secondary grammar evaluates the inline lexical contents. The evaluation of tree-sitter-markdown against the MarkText requirements is overwhelmingly positive:

- **Lossless CST:** It generates a pure Concrete Syntax Tree. Every byte, including whitespace, punctuation, and markers, is represented as a distinct node or token (e.g., `word_token`, `path_like_token`, `terminator`).
- **Incremental Reparse:** Tree-sitter natively supports $O(1)$ or $O(\log N)$ tree modifications. By passing the previous CST and the byte-edit ranges from the keystroke event, the engine reuses unmodified nodes and only reparses the altered text segment.
- **WASM and Electron:** web-tree-sitter is an industry standard for WASM embedding. Crucially, the CST remains entirely within the WASM heap. The JavaScript thread receives a lightweight Tree object containing a pointer. When Vue needs to render the viewport, it queries the WASM memory directly for the nodes currently visible on the screen, completely circumventing the JSON serialization bottleneck that would otherwise cripple performance.
- **GFM, Math, & YAML:** Implementations natively handle GFM pipe tables, task lists, YAML/TOML frontmatter, and LaTeX math. tree-sitter-quarto expands on this with first-class support for cross-references, executable cells, and complex generic divs.
- **Error Tolerance:** Mid-edit states naturally resolve to ERROR nodes in the Tree-sitter CST without halting the parser, leaving the surrounding document perfectly intact.

The primary challenge with Tree-sitter is implementing the intrinsic CriticMarkup requirement. Because a secondary parsing pass is forbidden, the development team must fork the tree-sitter-markdown (or tree-sitter-quarto) C/JavaScript grammar files (`grammar.js`) to natively define CriticMarkup syntax. To solve the multi-block CriticMarkup dilemma, `{++` and `++}` must be defined as both inline lexical tokens within the inline grammar and as generic block containers within the block grammar. Tree-sitter's GLR algorithm is specifically designed to handle this type of grammatical ambiguity, allowing the engine to recognize an addition spanning multiple paragraphs as a parent block containing paragraph children. Once the `grammar.js` is modified, the new Rust/WASM parser can be generated via the standard tree-sitter generate tooling.

Rowan-Based and Typst Architectures (Rust-Analyzer Paradigm)

rowan is a library developed by the rust-analyzer team for building lossless, error-resilient syntax trees using a Red-Green tree architecture. Green trees are immutable, position-independent, and heavily deduplicated in memory. Red trees act as a localized facade over the Green trees, providing absolute byte offsets and parent pointers for the editor to utilize. Typst utilizes a very similar bespoke incremental parsing architecture that tracks byte-level spans with extreme efficiency.

A rowan-based Markdown parser represents the theoretical holy grail of WYSIWYG parsing: infinite error tolerance, fully lossless round-tripping, sub-millisecond incremental updates via Green tree node sharing, and pure, idiomatic Rust execution. If a user deletes a character in a paragraph, a rowan parser merely allocates a new Green node for that specific word, a new Green node for the paragraph, and reuses the pointers for every other chapter and block in the document.

However, the availability of such an engine is non-existent. While theoretical implementations and discussions exist—such as issue #73 in the rowan repository discussing root node offsets for Markdown files—there is no off-the-shelf, CommonMark-compliant, production-ready rowan Markdown parser available. Choosing this architecture would require the MarkText development team to author an entire CommonMark and GFM parser from scratch. While technically superior in its memory model for pure Rust applications, the engineering cost of re-implementing the CommonMark specification—with its notoriously complex emphasis parsing, HTML block edge cases, and backtracking heuristics—is astronomical compared to adopting a pre-existing solution.

Ecosystem Factor	Extensibility for CriticMarkup	GFM & Math Support	WASM Binding Maturity	License & Health
pulldown-cmark	Low (Core modification needed)	Excellent	High (Community wrappers)	MIT, highly active
comrak	Low (C-port architecture)	Excellent (1:1 GitHub)	Excellent (comrak-wasm)	BSD-2-Clause, active
markdown-rs	Very Low (Monolithic)	Excellent	Excellent (mdxjs-rs)	MIT, active
markdown-it-rs	High (Plugin architecture)	Partial	Low	MIT, Alpha/Beta status
tree-sitter-markdown	High (Fork grammar.js)	Excellent	Excellent (web-tree-sitter)	MIT, highly active
rowan / Typst	Infinite (Scratch built)	None (Requires building)	N/A	MIT, stable frameworks

Strategic Synthesis and Conclusions

The architectural requirement to replace the "document-core" TypeScript engine with a Rust-based, source-authoritative, incrementally reparsing engine containing intrinsic CriticMarkup narrows the field decisively. Event stream and AST-based parsers fundamentally lack the necessary topological data structures (CST) and dynamic memory sharing mechanisms to function efficiently as the core of a sandboxed, modern WYSIWYG editor.

Disqualified Candidates

No single candidate can serve as a genuine, unmodified wholesale replacement for the custom engine. The following candidates are explicitly disqualified from serving as a foundational architecture:

1. **pulldown-cmark**: Unsuitable due to its event-stream nature, destructive formatting (non-lossless), and total lack of incremental reparsing. Modifying it for intrinsic CriticMarkup would require a fundamental rewrite of the state machine.
2. **comrak**: Generates a standard AST, permanently stripping the vital syntax trivia needed for WYSIWYG source mapping. Operates only in a batch parsing mode, precluding keystroke-level efficiency.
3. **markdown-rs (woorm)**: While theoretically brilliant at tracking byte provenance via tokens, its monolithic design prevents external extension, and it suffers from documented asymptotic complexity bugs that cripple large-document performance.
4. **markdown-it.rs**: Remains too immature (alpha state), utilizes a standard AST rather than a CST, and entirely lacks incremental reparsing capabilities.
5. **jotdown / carve**: Both operate on different, heavily opinionated specifications (Djot and Carve) and fail the strict CommonMark and GFM compliance requirement.

The Recommended Foundation: tree-sitter-markdown

tree-sitter-markdown (and its highly advanced derivative, tree-sitter-quarto) is the only existing candidate capable of serving as a realistic foundation for the MarkText editor engine.

This architecture natively generates a Concrete Syntax Tree, preserving all exact byte-spans, whitespace, and structural trivia demanded by the losslessness requirement. It possesses out-of-the-box incremental reparsing capabilities using tree edit graphs, ensuring near-zero latency on every keystroke. Its WASM bindings are mature and actively circumvent the V8 serialization bottleneck. Furthermore, it inherently possesses robust error tolerance, leaving incomplete node branches intact without panicking or dropping data.

Required Custom Engineering: The development team cannot use the crate entirely unmodified due to the strict intrinsic CriticMarkup requirement. The precise engineering work required involves forking the tree-sitter-markdown C/JavaScript grammar files. The team must inject CriticMarkup syntax rules directly into the GLR parsing tables. To handle the multi-block concurrency issue in a single pass, CriticMarkup boundaries must be defined ambiguously as both inline lexical elements and as block-level structural containers. Once the grammar is updated, the Rust WASM engine is recompiled, yielding a custom, high-performance, incremental CST parser that perfectly aligns with MarkText's architectural parameters.

Should modifying the C-based grammar files of Tree-sitter prove untenable, the only remaining alternative is to build a bespoke engine utilizing the rowan crate's Red-Green tree framework. While this would provide absolute control over the parser and flawless Rust integration, it constitutes building a highly complex CommonMark parser entirely from scratch—an undertaking requiring vast engineering resources and protracted development timelines. Thus, leveraging and extending a Tree-sitter implementation remains the optimal strategic pathway.

Works cited

1. pulldown-cmark/pulldown-cmark: An efficient, reliable parser for CommonMark, a standard

dialect of Markdown - GitHub, <https://github.com/pulldown-cmark/pulldown-cmark> 2. GitHub - woorm/markdown-rs: CommonMark compliant markdown parser in Rust with ASTs and extensions, <https://github.com/woorm/markdown-rs> 3. Any crate that is able to parse GitHub CommonMark to HTML? : r/rust - Reddit, https://www.reddit.com/r/rust/comments/kd363q/any_crate_that_is_able_to_parse_github_commonmark/ 4. Add the ability to make a SyntaxNode which is offset by a certain amount from the source code · Issue #73 · rust-analyzer/rowan - GitHub, <https://github.com/rust-analyzer/rowan/issues/73> 5. tree-sitter-quarto/docs/comparison.md at main - GitHub, <https://github.com/ck37/tree-sitter-quarto/blob/main/docs/comparison.md> 6. Marko Editor is a simple WYSIWYG editor for note taking. - GitHub, <https://github.com/mmMike/marko-editor> 7. GitHub - markup-carve/carve: A post-Markdown lightweight markup language with visual mnemonics and human-centered design., <https://github.com/markup-carve/carve> 8. ophi-dev/tree-sitter-markdown-text - GitHub, <https://github.com/ophidiarium/tree-sitter-markdown-text> 9. Tree-sitter parser for Quarto Markdown (.qmd files) with first-class support for executable cells, chunk options, and cross-references. Editor-focused for syntax highlighting, validation, and navigation. - GitHub, <https://github.com/ck37/tree-sitter-quarto> 10. 2389 Research, Inc - GitHub, <https://github.com/2389-research> 11. casile - crates.io: Rust Package Registry, <https://crates.io/crates/casile> 12. pulldown-cmark - GitHub, <https://github.com/pulldown-cmark> 13. pulldown-cmark-mdcat - crates.io: Rust Package Registry, <https://crates.io/crates/pulldown-cmark-mdcat> 14. Support for Description Lists · Issue #67 · pulldown-cmark/pulldown-cmark - GitHub, <https://github.com/google/pulldown-cmark/issues/67> 15. Convert pulldown-cmark Events back to the string they were parsed from - GitHub, <https://github.com/Byron/pulldown-cmark-to-cmark> 16. hellux/jotdown: A Djot parser library - GitHub, <https://github.com/hellux/jotdown> 17. GitHub - kivikakk/comrak: CommonMark + GFM compatible Markdown parser and renderer, <https://github.com/kivikakk/comrak> 18. Extending markdown-rs #52 - GitHub, <https://github.com/woorm/markdown-rs/discussions/52> 19. TypeScript and WebAssembly bindings for Comrak, the Markdown-to-HTML renderer written in Rust. - GitHub, <https://github.com/nberlette/comrak-wasm> 20. Python bindings for the Comrak Rust library, a fast CommonMark/GFM parser - GitHub, <https://github.com/lmmx/comrak> 21. markdown-it-pyrs - Python Simple Repository Browser, <https://simple-repository.app.cern.ch/project/markdown-it-pyrs> 22. markdown-it js library rewritten in rust - GitHub, <https://github.com/markdown-it-rust/markdown-it> 23. Titus woorm - GitHub, <https://github.com/woorm> 24. micromark/micromark: small, safe, and great commonmark (optionally gfm, mdx) compliant markdown parser - GitHub, <https://github.com/micromark/micromark> 25. Mdxjs Rs - AI Agent Skill | Open Agent Skill, <https://www.openagentskill.com/skills/woorm-mdxjs-rs> 26. Performance: larger MDX files are unmanagably slow to parse · Issue #113 · woorm/markdown-rs - GitHub, <https://github.com/woorm/markdown-rs/issues/113> 27. quarto-markdown/CLAUDE.md at main - GitHub, <https://github.com/quarto-dev/quarto-markdown/blob/main/CLAUDE.md>